

# P1381R0

## Reference capture of structured bindings

New Proposal, 2018-11-26

**This version:**

<https://wg21.link/p1381r0>

**Author:**

Nicolas Lesser (blitzrakete[at]gmail[dot][com])

**Audience:**

EWG

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

### Abstract

Analysis of reference capture of structured bindings.

### Table of Contents

#### 1 Introduction

#### 2 Problem

#### 3 What can we do?

### § 1. Introduction

In San Diego, EWG decided to allow capture by value for structured bindings. The following polls were taken:

1. Capturing structured bindings by value either by default capture or by name capture always copies the value of the structured binding

| SF | F | N | A | SA |

| 12 | 14 | 1 | 2 | 3 |

2. Capturing structured bindings by value, either by default capture or by named capture, always copies whole/underlying object

| SF | F | N | A | SA |

| 2 | 3 | 5 | 6 | 8 |

3. Capturing structured bindings by value by default capture captures the underlying object and by named capture only the named bindings.

| SF | F | N | A | SA |

| 0 | 3 | 3 | 8 | 14 |

The first poll got consensus, thus allowing:

```
struct Foo { int a; };
void f() {
    auto[a] = Foo();
    auto f1 = [a] { return a; }; // ok, captures 'a'
    auto f2 = [=] { return a; }; // ok, captures 'a'
}
```

However, there were concerns about capturing structured bindings by reference.

## § 2. Problem

Bit-fields.

```

struct Foo { int a : 1; };
void f() {
    auto[a] = Foo();
    // 'a' is a bit-field!

    auto f1 = [&a] { return a; }; // capturing a bit-field by reference?!?
}

```

Since EWG agreed on the behavior of by-value capture of structured bindings, it would make sense to allow by-reference capture to behave the same, i.e. that only a reference to the bindings capture are formed and not to the underlying object.

### § 3. What can we do?

Bit-fields have some funny behavior. There isn't a type for them for example. One important difference is that you can't take the address of a bit-field, so it follows that a reference to a bit-field is also ill-formed.

```

struct Foo { int a : 1; };
void f(int&);

int main() {
    Foo foo;
    f(foo.a); // ill-formed, reference to bit-field
}

```

As such, I'm proposing that reference capture of a bit-field remains ill-formed. I see no reason why this proposal should change this.

```

struct Foo { int a : 1; int b; };

int main() {
    auto[a, b] = Foo();

    auto f1 = [&] { return a; }; // still ill-formed with this proposal
    auto f2 = [&a = a] { return a; }; // always was ill-formed; no change
    auto f3 = [&a] { return a; }; // still ill-formed with this proposal

    auto g1 = [&] { return b; }; // previously ill-formed; well-formed v
    auto g2 = [&b = b] { return b; }; // always was well-formed; no change
    auto g3 = [&b] { return b; }; // previously ill-formed; well-formed v
}

```

Every other structured binding has a "valid" type (a type which we can reference); bit-fields are the only exception to this rule.